CERTIK

8lends - Audit 2

Preliminary Comments

CertiK Assessed on Nov 13th, 2025

CertiK Assessed on Nov 13th, 2025

8lends - Audit 2

These preliminary comments were prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
DeFi	EVM Compatible	Formal Verification, Manual Review, Static Analysis

LANGUAGE	TIMELINE
Solidity	Preliminary comments published on 11/13/2025

Vulnerability Summary

20	0	0	0	0	20
Total Findings	Resolved	Partially Resolved	Acknowledged	Declined	Pending

2	Centralization	2 Pending	Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
1	Major	1 Pending	Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
4	Medium	4 Pending	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
10	Minor	10 Pending	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
3	Informational	3 Pending	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.
0	Discussion		The impact of the issue is yet to be determined, hence requires further clarifications from the project team.

TABLE OF CONTENTS | 8LENDS - AUDIT 2

CODEBASE | 8LENDS - AUDIT 2

Repository

https://github.com/8lends-org/8lends-smart-contracts/tree/05d8b3b2b6598f66ae5df884d451d8d4bb58449f

AUDIT SCOPE | 8LENDS - AUDIT 2

8lends-org/8lends-smart-contracts
contracts/Fundraise.sol
contracts/ManagerRegistry.sol
contracts/RewardSystem.sol
contracts/Token.sol
contracts/lib/MerkleProof.sol
contracts/lib/SignerLib.sol
contracts/Treasury.sol

APPROACH & METHODS | 8LENDS - AUDIT 2

This audit was conducted for 8lends to evaluate the security and correctness of the smart contracts associated with the 8lends - Audit 2 project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Formal Verification, Manual Review, and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

FINDINGS | 8LENDS - AUDIT 2

20	0	2	1	4	10	3	0
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational	Discussion

This report has been prepared for 8lends to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 20 issues were identified. Leveraging a combination of Formal Verification, Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
8A2-01	**Centralized Control Of Contract Upgrade**	**Centralization**	**Centralization**	**Pending**
8A2-02	**Centralization Related Risks**	**Centralization**	**Centralization**	**Pending**
8A2-03	Lack Of Collateralization Exposes Investors To Repayment Risk	Design Issue	Major	Pending
8A2-04	Reward Amount Is Derived From Manipulable Uniswap Spot Quote	Financial Manipulation	Medium	Pending
8A2-05	Ineffective Slippage Tolerance Allows Sandwich Attack	Logical Issue	Medium	Pending
8A2-06	Inconsistent Token Assumptions Lead To Bonus Exploitation	Inconsistency	Medium	Pending
8A2-07	`RewardSystem` Bypasses Token's Minting Restriction	Inconsistency	Medium	Pending
8A2-08	Signature Malleability Of `ecrecover`	Volatile Code	Minor	Pending
8A2-09	Zero Address `trustedSigner` Allows Arbitrary Data To Be Fraudulently Signed	Volatile Code	Minor	Pending
8A2-10	Approved Address Can Approve Other Addresses	Design Issue	Minor	Pending
8A2-11	Missing Emit Events (Access Control Parameter)	Inconsistency	Minor	Pending

ID	Title	Category	Severity	Status
8A2-12	Missing Zero Address Validation	Volatile Code	Minor	Pending
8A2-13	`enableBuyingForever` Does Not Prevent Future Disabling	Inconsistency	Minor	Pending
8A2-14	Inconsistent State Update For Merkle Root	Inconsistency	Minor	Pending
8A2-15	Inconsistent Soft Cap Evaluation	Inconsistency	Minor	Pending
8A2-16	Missing Project Initialization Checks	Access Control	Minor	Pending
8A2-17	Inconsistent Token Decimals Assumption	Volatile Code	Minor	Pending
8A2-18	Incompatibility With Deflationary Tokens (Non-Standard ERC20 Token)	Volatile Code	Informational	Pending
8A2-19	`burnPercentage` Is Misleading Named	Coding Style	Informational	Pending
8A2-20	Overly Restrictive Access Control For `makeRepayment()`	Design Issue	Informational	Pending

8A2-01 Centralized Control Of Contract Upgrade

Category	Severity	Location	Status
Centralization	Centralization		Pending

Description

In the contracts `Fundraise`, `ManagerRegistry`, `RewardSystem` and `Treasury`, the role `owner` has the authority to update the implementation contract behind the proxy contract.

Any compromise to the `owner` account may allow a hacker to take advantage of this authority and change the implementation contract which is pointed by proxy and therefore execute potential malicious functionality in the implementation contract.

Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature (⅔, ⅗) wallet can be used to prevent a single point of failure due to a private key compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

Short Term:

A combination of a time-lock and a multi signature (⅔, ⅗) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;
  AND
- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
  AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
  AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
  OR
- Remove the risky functionality.

Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.

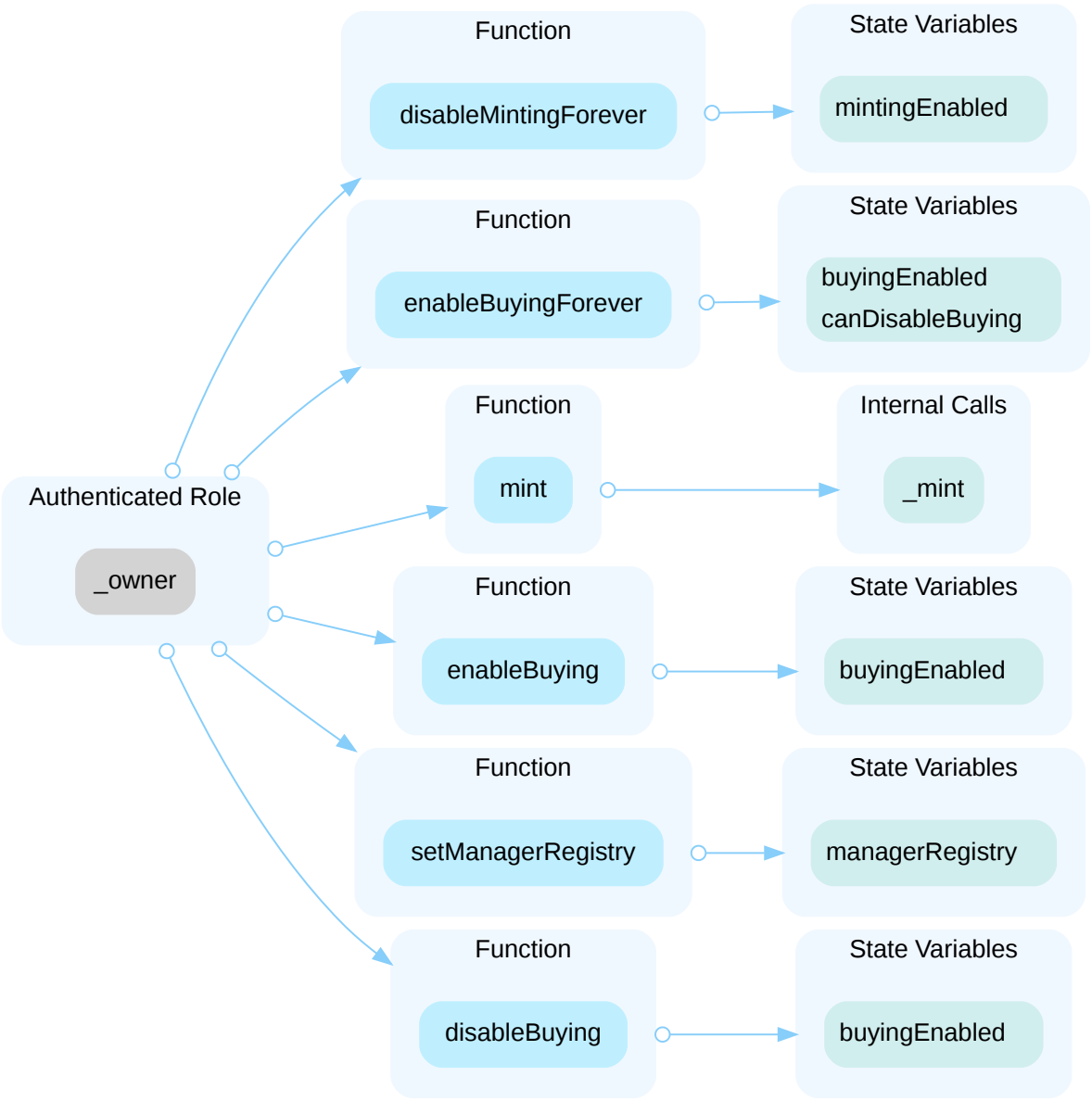
8A2-02 Centralization Related Risks

Category	Severity	Location	Status
Centralization	**Centralization**		**Pending**

Description

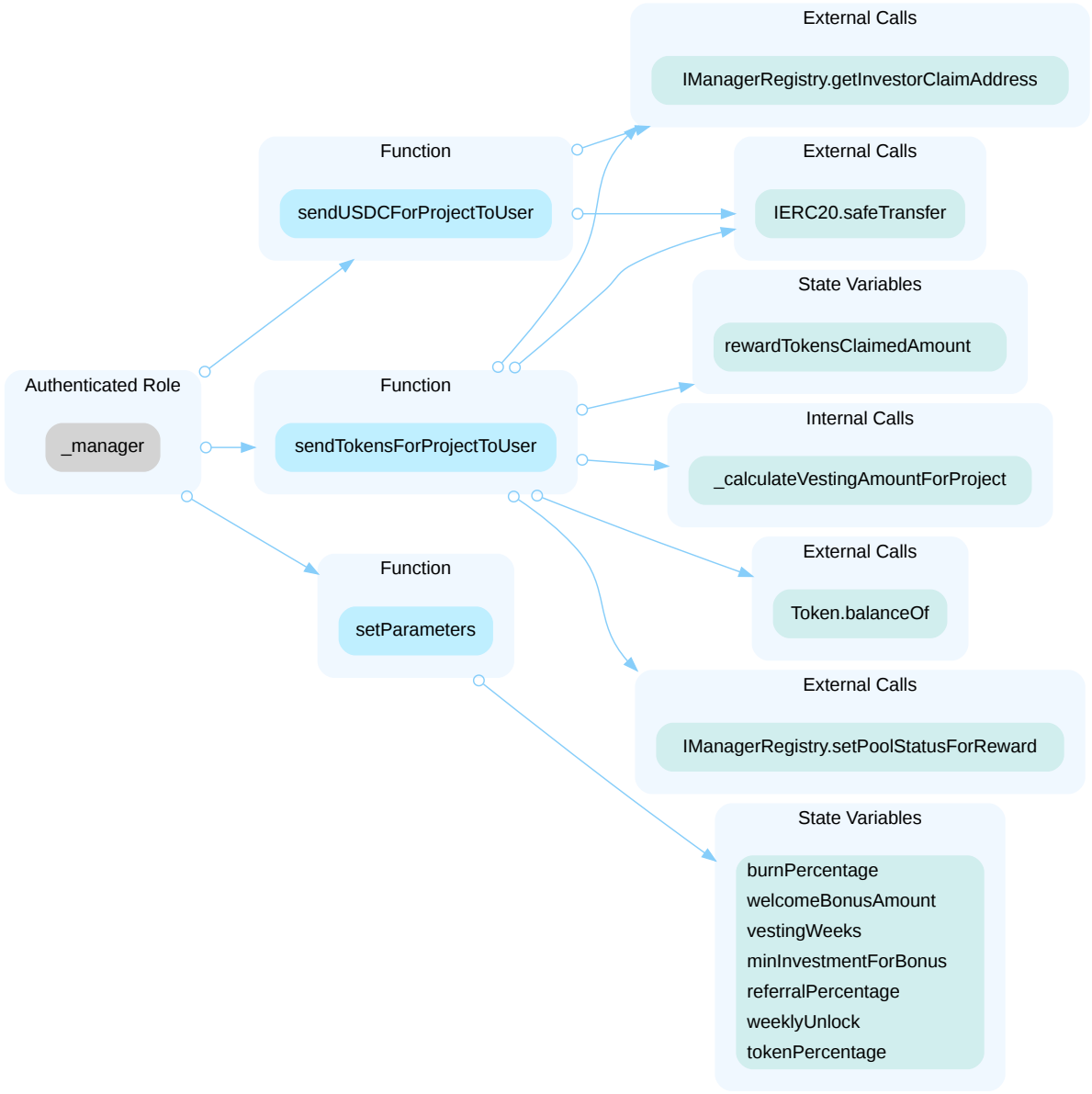
In the contract `Token` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and

- `enableBuying()` : Enables the public buying and transfer of the token.
- `disableBuying()` : Disables the public buying and transfer of the token.
- `enableBuyingForever()` : Permanently enables the public buying of the token, removing the ability to disable it later.
- `disableMintingForever()` : Permanently disables owner's ability to mint new tokens.
- `mint()` : Mints a specified amount of new tokens to a given address.
- `setManagerRegistry()` : Sets the address of the ManagerRegistry contract.



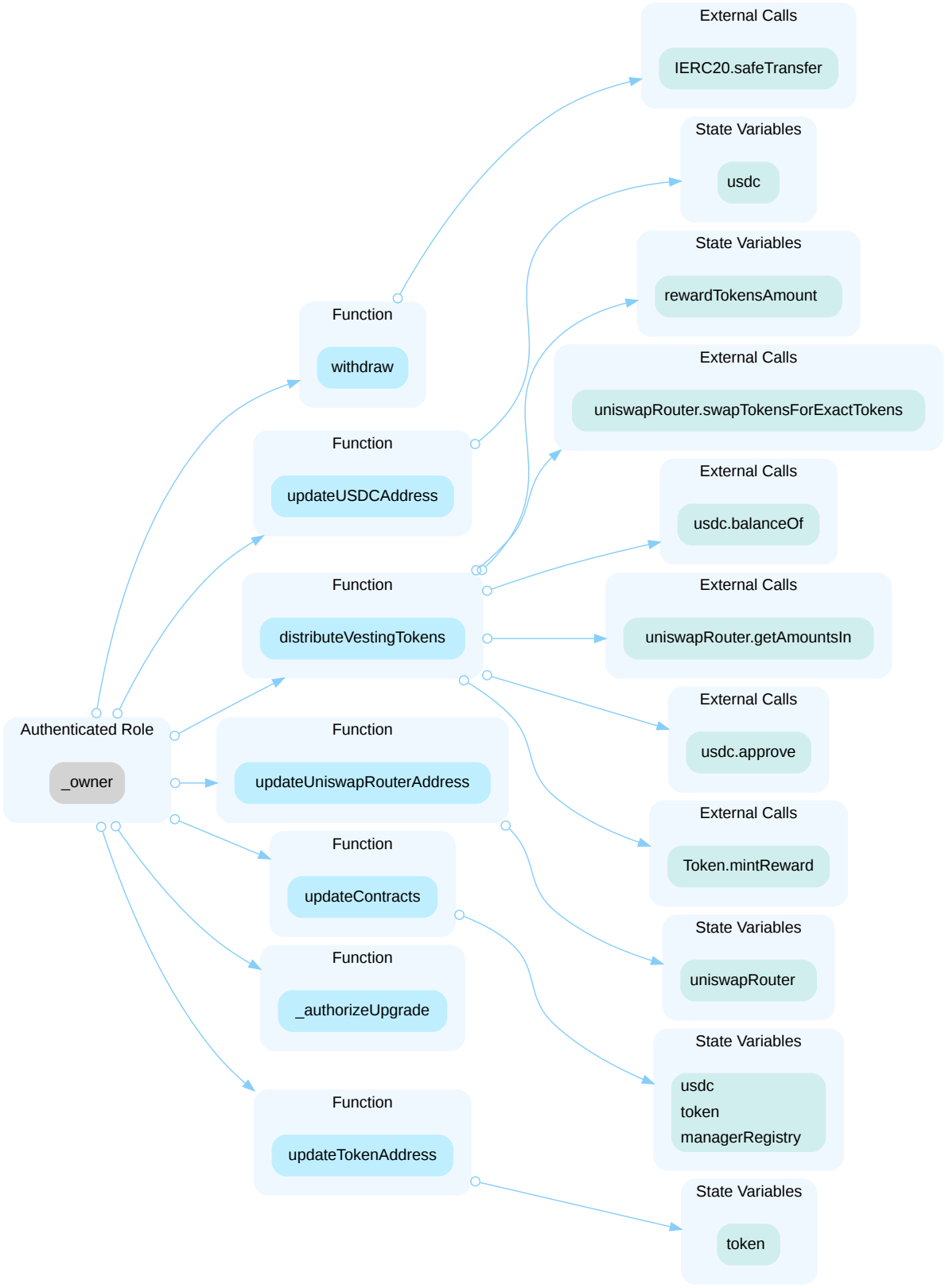
In the contract `RewardSystem` the role `_manager` has authority over the functions shown in the diagram below. Any compromise to the `_manager` account may allow the hacker to take advantage of this authority and

- `sendUSDCForProjectToUser()` : Sends USDC rewards on behalf of a specified user.
- `sendTokensForProjectToUser()` : Sends 8LNDS token rewards on behalf of a specified user.
- `setParameters()` : Adjusts key reward system parameters like referral percentages, bonus amounts, and vesting schedules.



In the contract `RewardSystem` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and

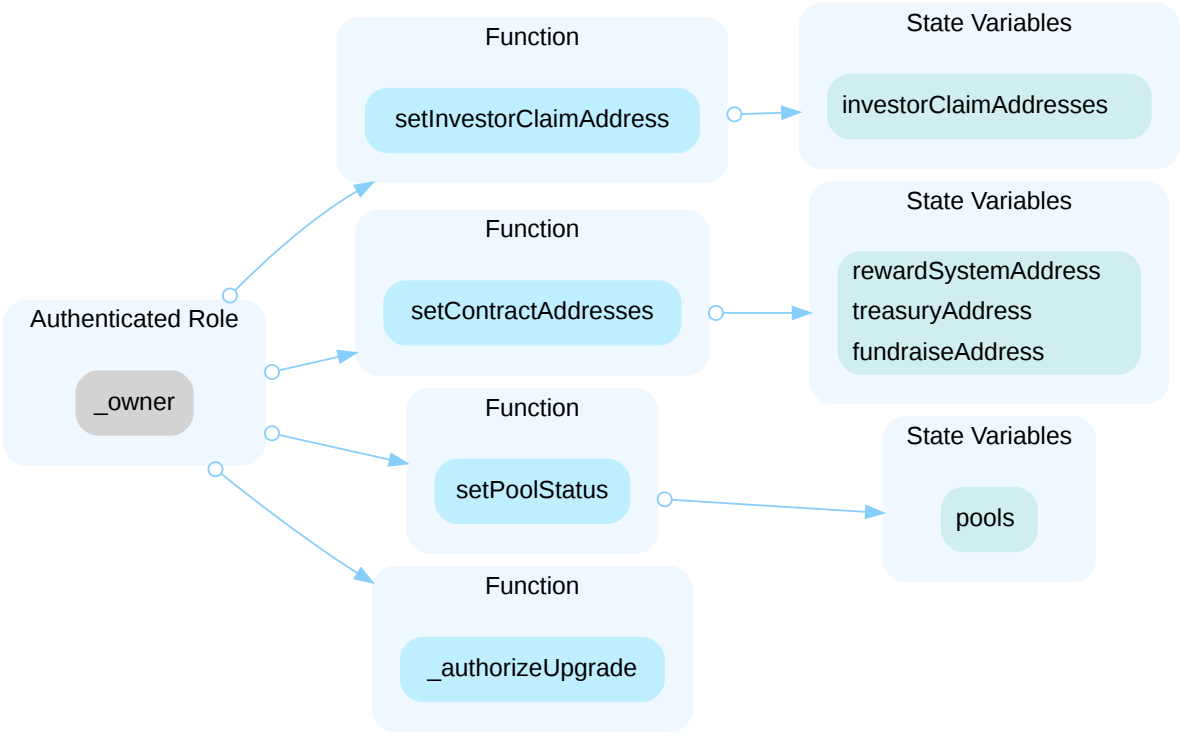
- `updateContracts()` : Updates the addresses of the manager registry, token, and USDC contracts.
- `_authorizeUpgrade()` : Authorizes a contract upgrade to a new implementation.
- `updateUSDCAddress()` : Updates the address of the USDC contract.
- `updateTokenAddress()` : Updates the address of the 8LNDS token contract.
- `updateUniswapRouterAddress()` : Updates the address of the Uniswap Router contract.
- `distributeVestingTokens()` : Distributes vesting tokens to a list of users, with the option to mint new tokens or buy them from a pool.
- `withdraw()` : Withdraws any specified ERC20 token from the contract to a chosen recipient.



In the contract `ManagerRegistry` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and

- `setManagerStatusBatch()` : Allows an owner to add or remove multiple managers at once.
- `setManagerStatus()` : Allows an owner to add or remove a single manager.
- `setPoolStatus()` : Adds or removes a pool address.

- `setContractAddresses()` : Sets the addresses for the reward system, fundraise, and treasury contracts.
- `setInvestorClaimAddress()` : Sets a specific address to receive payouts for an investor.
- `_authorizeUpgrade()` : Authorizes a contract upgrade to a new implementation.

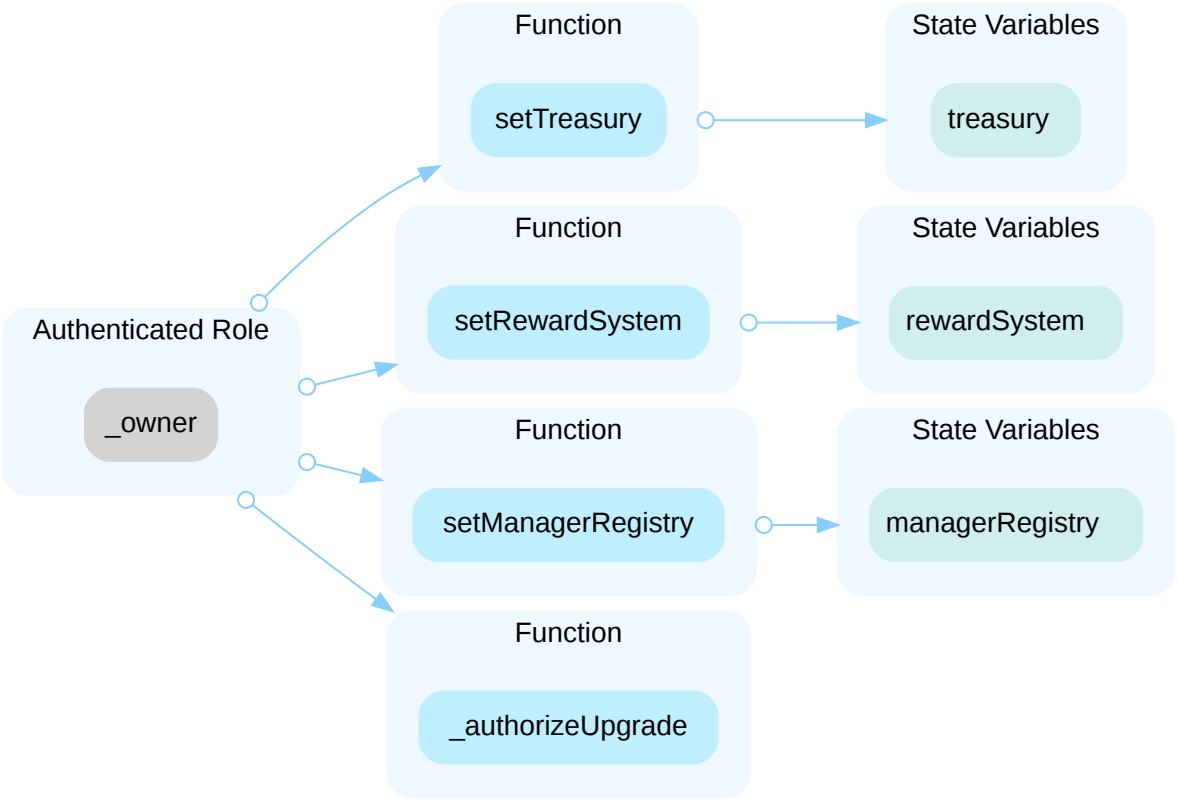


In the contract `ManagerRegistry` the role `manager` has authority over the functions below. Any compromise to the `manager` account may allow the hacker to take advantage of this authority and

- `setManagerStatusBatch()` : Allows a manager to add or remove other managers.
- `setManagerStatus()` : Allows a manager to add or remove another manager.

In the contract `Fundraise` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and

- `setManagerRegistry()` : Updates the address of the ManagerRegistry contract.
- `setTreasury()` : Updates the address of the Treasury contract.
- `setRewardSystem()` : Updates the address of the RewardSystem contract.
- `_authorizeUpgrade()` : Authorizes a contract upgrade to a new implementation.

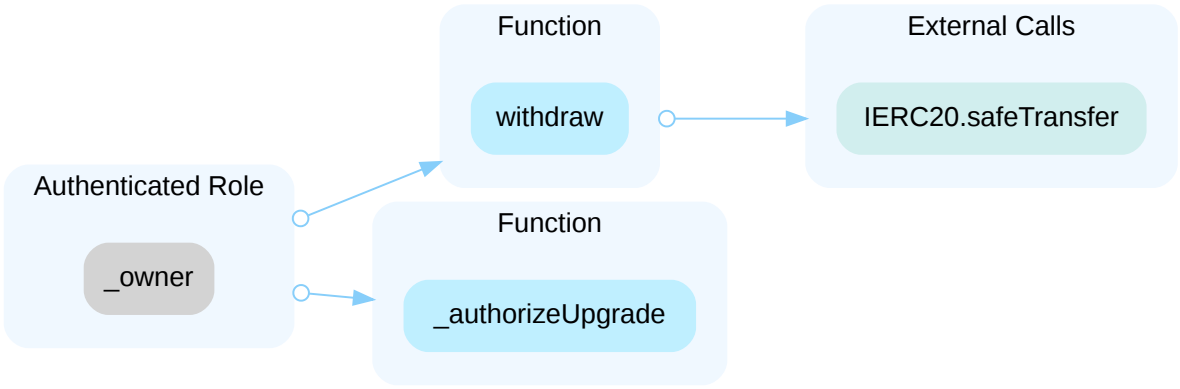


In the contract `Fundraise` the role `manager` has authority over the functions below. Any compromise to the `manager` account may allow the hacker to take advantage of this authority and

- `cancelProject()` : Cancels a project before it is funded.
- `transferFundsToBorrower()` : Triggers the transfer of collected funds to the borrower.
- `makeRepayment()` : Allows a manager to make a repayment on behalf of the borrower.
- `claim()` : Allows a manager to claim investment returns on behalf of an investor.
- `createProject()` : Creates a new fundraising project.
- `moveProjectStage()` : Manually advances the stage of a project.
- `setProject()` : Updates the parameters of an existing project.
- `setWhitelist()` : Sets or updates the Merkle root for a project's whitelist.
- `setTrustedSigner()` : Updates the address of the trusted signer used for signature verification.

In the contract `Treasury` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and

- `withdraw()` : Withdraws any specified ERC20 token from the contract to a chosen recipient.
- `_authorizeUpgrade()` : Authorizes a contract upgrade to a new implementation.



Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.

  OR

- Remove the risky functionality.

8A2-03 Lack Of Collateralization Exposes Investors To Repayment Risk

Category	Severity	Location	Status
Design Issue	Major	contracts/Fundraise.sol: 270	Pending

Description

The `Fundraise` contract serves as a platform for borrowers to raise capital from investors. The core assumption of such a lending-borrowing system is that borrowers will repay their debts, allowing investors to claim their principal and earnings.

However, a fundamental flaw in the contract's design is the complete absence of collateral requirements for borrowers. Unlike traditional lending protocols, the `Fundraise` contract does not mandate borrowers to stake any assets as collateral that could be liquidated to cover investor losses in case of non-repayment.

This design choice transforms the loan into an unsecured debt, where the borrower's decision to `makeRepayment()` is entirely voluntary. The `project.investorInterestRate` is not calculated based on the actual duration for which the funds were borrowed. There are no on-chain incentives or penalties, beyond potential off-chain reputation, to ensure that the borrower repays the funds. Consequently, if a borrower chooses not to repay, investors have no recourse within the smart contract system to recover their invested capital. The contract effectively acts as a trust-based lending platform without the cryptographic guarantees typically associated with blockchain-based financial instruments.

Recommendation

Before funds are disbursed to the borrower, the contract should require the borrower to escrow sufficient collateral (e.g., another cryptocurrency, or even NFTs) within the `Fundraise` contract or a linked collateral management contract.

Implement logic to calculate the interest rate based on the actual duration for which the funds were borrowed.

Implement logic to allow for the liquidation of the collateral if the borrower fails to `makeRepayment()` by a specified deadline.

8A2-04 Reward Amount Is Derived From Manipulable Uniswap Spot Quote

Category	Severity	Location	Status
Financial Manipulation	Medium	contracts/RewardSystem.sol: 221, 514	Pending

Description

`recordInvestment` computes the investor's token reward as `tokensAmount = uniswapRouter.getAmountsOut((_amount * tokenPercentage) / 1e6, [USDC, token])[1]`, and immediately credits:

- projectReferrals[_user][_projectId].totalRewardsTokens += tokensAmount
- rewardTokensAmount[_projectId] += tokensAmount

`getAmountsOut` returns a spot-price quote based solely on current AMM reserves. The function does not use a TWAP or oracle tying `tokensAmount` to the real market value of `_amount`.

Anyone investing through the normal fundraise flow can execute a transaction that first distorts the USDC/token pool price, then triggers `recordInvestment`, and finally restores the price (e.g., via a flash swap). No privileged roles are required; the function trusts the manipulable `getAmountsOut` result.

Scenario

1. Prepare a contract that can perform a Uniswap V2 flash swap or otherwise trade in the USDC/token pair.
2. Flash swap a large amount of token from the USDC/token pair, then immediately swap those tokens into the same pair, selling token for USDC. This increases token reserve and decreases USDC reserve, sharply lowering token's price (i.e., increasing tokens per USDC).
3. In the same transaction, call the normal fundraise investment method that triggers RewardSystem.recordInvestment for your address with:
    - _user = your address
    - _amount = your chosen USDC investment (≥ 0; choose any amount; larger amount amplifies effect)
    - _inviter = any non-zero address (optionally an alternate wallet to also receive the referral USDC)
    - _projectId = target project
    - During this call, getAmountsOut is evaluated against the distorted reserves, returning an exaggerated tokensAmount. The function immediately records this inflated amount to your totalRewardsTokens and to rewardTokensAmount for the project.
4. After recordInvestment returns, swap USDC back to token to restore reserves and repay the flash swap. The pool price returns to normal, but the recorded, inflated reward remains.
5. Later, when the project proceeds, those recorded tokens (rewardTokensAmount) are minted/used for vesting distribution, letting you claim vastly more tokens than intended.

Recommendation

Replace the use of `getAmountsOut` (spot quote) with a price feed that is resistant to manipulation, such as a time-weighted average price (TWAP) or a trusted oracle. Ensure that all reward calculations are based on price data that cannot be manipulated within a single transaction.

8A2-05 | Ineffective Slippage Tolerance Allows Sandwich Attack

Category	Severity	Location	Status
Logical Issue	Medium	contracts/Fundraise.sol: 221~236, 520~529	Pending

Description

The `activateProjectRewards()` and `distributeVestingTokens()` functions attempt to implement a 1% slippage tolerance. This is done by calculating `maxUSDNeeded` as 101% of the USDC amount determined by `uniswapRouter.getAmountsIn()` and passing it as `amountInMax` to the `uniswapRouter.swapTokensForExactTokens()` call.

However, the `UniswapV2Router01::swapTokensForExactTokens()` function, which is called by the `RewardSystem` contract, does not use `amountInMax` as the actual amount to transfer. Instead, it calculates the exact amount of input tokens required (`amounts[0]`) using `UniswapV2Library.getAmountsIn()` and then performs the transfer using this exact amount. The `amountInMax` parameter is only used in a `require` statement to ensure that the exact amount needed (`amounts[0]`) does not exceed the maximum allowed (`amountInMax`).

As above two functions are in the same transaction, the transaction will succeed, and the contract will pay the exact amount of USDC required at the time of execution, without the intended 1% buffer being utilized to protect against adverse price changes. This means that the `RewardSystem` contract effectively has no slippage protection for the `activateProjectRewards` function.

The `activateProjectRewards()` function mints accumulated rewards tokens and buys the same amount of token from the DEX pool. As the `rewardTokensAmount` of a project could be a large amount, it is highly susceptible to significant price impact on the DEX. This vulnerability creates an opportunity for MEV bots to execute sandwich attacks, front-running the transaction to profit at the expense of the project and its users.

If the token used in the DEX pair `token-USDC` is `8LNDS` , enabling the token "buying" function will cause all remaining OPEN and PreFunded projects to be impacted by this issue.

Scenario

1. A legitimate call to `RewardSystem::activateProjectRewards()` is broadcasted to the blockchain network. This transaction, which includes a large buy order on a DEX, enters the mempool.
2. An MEV bot monitors the mempool and identifies this transaction. Recognizing the potential for significant price impact due to the large `rewardTokensAmount` , the bot constructs two new transactions:
    - **Front-run (Buy):** The bot places a buy order for the token that `activateProjectRewards()` intends to buy, executing its transaction just before the `activateProjectRewards()` transaction. This initial buy order drives up the price of the token.
    - **Sandwich (Sell):** After the `activateProjectRewards()` transaction executes (which now buys at an artificially inflated price, causing further price increase), the bot immediately places a sell order for the token it just bought,

executing its transaction just after `activateProjectRewards()`. This sell order profits from the price increase caused by both the bot's front-run and the `activateProjectRewards()` transaction.

3. The `activateProjectRewards()` transaction ends up buying tokens at a significantly higher price than it would have otherwise, incurring a substantial loss due to the price impact and the MEV bot's manipulation. The MEV bot captures the value that would otherwise have been lost by the `RewardSystem` contract due to price impact.

Recommendation

For `activateProjectRewards()`, consider revisiting the current design, the sequence of actions: mint -> buy -> burn the same amount of tokens, could be simplified to a direct distribute of the equivalent value of USDC to investors.

For `distributeVestingTokens()`, consider removing the `_buyFromPool` feature.

8A2-06 | Inconsistent Token Assumptions Lead To Bonus Exploitation

Category	Severity	Location	Status
Inconsistency	Medium	contracts/Fundraise.sol: 53; contracts/RewardSystem.sol: 108, 154	Pending

Description

The `RewardSystem` contract includes a mechanism to provide a welcome bonus (30 USDC). This bonus is contingent on meeting a `minInvestmentForBonus` threshold:

```
108  minInvestmentForBonus = 1000e6; // Minimum 1000 USDC for bonus
```

The implicit assumption behind this threshold appears to be that the investment token will be a token with 6 decimals, ensuring that the minimum investment represents a meaningful monetary value.

However, the `Fundraise` contract, which manages the creation and parameters of fundraising projects, allows for the specification of an arbitrary `loanToken`. This means that a project can be configured to accept any ERC-20 token as its investment currency, not just USDC.

The vulnerability arises from the mismatch between the `RewardSystem`'s assumption about the value of `minInvestmentForBonus` (i.e., it's denominated in a high-value, stable token like USDC) and the `Fundraise` contract's flexibility in accepting any `loanToken`. If a project is created to accept a `loanToken` with a very low unit value (e.g., a meme coin or a token with 18 decimal places and a low price per token), a user can easily meet the `minInvestmentForBonus` numerical requirement with a very small actual monetary investment.

Additionally, inviter and investor rewards are also impacted, as their rewards (USDC or USDC value equivalent) are calculated based on the invested `loanToken` amount.

Scenario

1. A project creator deploys a `Fundraise` project that accepts a low-value token, "DogeTokens" (hypothetical, with a very low price, e.g., $0.0001 per token), as its `loanToken`.
2. The `RewardSystem` contract's `minInvestmentForBonus` is set to `1000e6`.
3. A user invests `1000e6` DogeTokens into this project.
4. Because the user's investment meets the `minInvestmentForBonus` numerical threshold, the `RewardSystem` awards the user a 30 USDC welcome bonus.
5. The user effectively pays a few cents to receive 30 USDC. This can be repeated multiple times, draining the bonus pool.

Recommendation

The simplest solution is to enforce that all `Fundraise` projects eligible for the bonus must accept USDC as their `loanToken`. This would require a check in the `Fundraise` contract's project creation logic.

If arbitrary `loanToken`s are to be supported, the `RewardSystem` must convert the `minInvestmentForBonus` into a standardized value (e.g., USD equivalent) using a reliable price oracle.

8A2-07 | `RewardSystem` Bypasses Token's Minting Restriction

Category	Severity	Location	Status
Inconsistency	Medium	contracts/RewardSystem.sol: 482	Pending

Description

The `Token::disableMintingForever()` function is designed to permanently prevent any further minting of tokens by the owner. This function, once called, sets a flag (`_mintingDisabled`) that should halt all owner minting operations. However, the `RewardSystem::distributeVestingTokens()` function, specifically when the `_buyFromPool` parameter is set to `false` , directly calls the `Token::mintReward()` function. This direct call bypasses the `_mintingDisabled` check within the `Token` contract, effectively allowing the `RewardSystem` 's owner to mint new tokens from air even after minting has been globally disabled in the `Token` contract. This contradicts the explicit design of the `Token` contract.

Recommendation

Consider removing the `_buyFromPool` parameter and the related token minting logic from the `RewardSystem::distributeVestingTokens()` function.

8A2-08 | Signature Malleability Of `ecrecover`

Category	Severity	Location	Status
Volatile Code	Minor	contracts/Fundraise.sol: 135	Pending

Description

The `ecrecover()` function is subject to signature malleability. Signature malleability is possible within the Elliptic Curve cryptographic system. An elliptic curve is symmetric on the `X` -axis, meaning two points can exist with the same `X` value. In the `r` ,'s', and `v` representations, this permits us to carefully adjust `s` to produce a second valid signature for the same `r` , thus breaking the assumption that a signature cannot be replayed in what is known as a replay-attack.

Recommendation

We suggest to consider the example in `ECDSA.sol` from the OpenZeppelin library to prevent signature malleability.

8A2-09 Zero Address `trustedSigner` Allows Arbitrary Data To Be Fraudulently Signed

Category	Severity	Location	Status
Volatile Code	Minor	contracts/Fundraise.sol: 137	Pending

Description

The `investUpdate` function is missing the zero address check for `trustedSigner`. If the signer is set as zero address, any malicious user is able to create a fraudulent signature which returns `address(0)` from ecrecover() function to bypass the signature check.

Recommendation

We recommend checking the `trustedSigner` is not `address(0)`.

8A2-10 Approved Address Can Approve Other Addresses

Category	Severity	Location	Status
Design Issue	Minor	contracts/ManagerRegistry.sol: 48	Pending

Description

The `setManagerStatus` function contains a permission-management issue. When an owner grants permissions or approval to another address (address B), this address B also gains the ability to approve other addresses with the same role. This can lead to unintended permission propagation and potential security risks.

Recommendation

Consider implementing a separate modifier to separate things apart.

8A2-11 | Missing Emit Events (Access Control Parameter)

Category	Severity	Location	Status
Inconsistency	Minor	contracts/RewardSystem.sol: 90~114, 351~358; contracts/Token.sol: 85~88	Pending

Description

The smart contract contains one or more state changes related to access control that do not emit events to communicate the changes outside the blockchain, which can lead to difficulties in tracking or verifying these changes and may affect the contract's transparency and auditability.

```
87          managerRegistry = _managerRegistry;
```

```
87          managerRegistry = _managerRegistry;
```

```
100          managerRegistry = _managerRegistry;
```

```
100          managerRegistry = _managerRegistry;
```

```
355          if (_managerRegistry != address(0)) managerRegistry = _managerRegistry;
```

```
355          if (_managerRegistry != address(0)) managerRegistry = _managerRegistry;
```

Recommendation

It is suggested to declare and emit corresponding events for all the essential state variables that are possible to be changed during runtime.

8A2-12 Missing Zero Address Validation

Category	Severity	Location	Status
Volatile Code	Minor	contracts/Fundraise.sol: 99~102, 416, 423, 429, 435; contracts/ManagerRegistry.sol: 75; contracts/RewardSystem.sol: 100~103, 470	Pending

Description

Addresses are not validated before assignment or external calls, potentially allowing the use of zero addresses and leading to unexpected behavior or vulnerabilities. For example, transferring tokens to a zero address can result in a permanent loss of those tokens.

Recommendation

It is recommended to add a zero-check for the passed-in address value to prevent unexpected errors.

8A2-13 `enableBuyingForever` Does Not Prevent Future Disabling

Category	Severity	Location	Status
Inconsistency	Minor	contracts/Token.sol: 50, 62	Pending

Description

Calling `enableBuyingForever()` sets `canDisableBuying = false`, implying buying can never be disabled afterward. However, `disableBuying()` ignores `canDisableBuying` and can still be called by the owner to turn buying off. This contradicts the "forever" guarantee and can mislead users: the owner could enable buying “forever” and later disable it, blocking buys and regular transfers (except to pools/reward systems).

Recommendation

Modify `disableBuying()` to respect the `canDisableBuying` flag, ensuring that once `enableBuyingForever()` is called, buying cannot be disabled by any party, including the owner. This enforces the intended "forever" guarantee and prevents misleading behavior.

8A2-14 | Inconsistent State Update For Merkle Root

Category	Severity	Location	Status
Inconsistency	Minor	contracts/Fundraise.sol: 138, 154, 158	Pending

Description

The `Fundraise::investUpdate()` function updates the Merkle root for project investors. The function includes a check (L154) that causes an early return if the `project.stage` is `ComingSoon` and the project has not yet started (`block.timestamp < project.startTime`). This check is intended to prevent any further processing or state changes related to investment if the project is not yet active.

Before this early return condition is evaluated, the function updates the `whitelistRoots` mapping with the provided `_rootHash` at L138. This means that even when the project is in the `ComingSoon` stage and the function is supposed to do nothing, the Merkle root for that project is still updated.

This creates an inconsistent state: the project is not yet active for investments, but its Merkle root, which dictates who invested, has already been modified.

Similarly, if `block.timestamp > project.openStageEndAt`, the same issue occurs.

Recommendation

It is recommended to modify the `_invest` function to include a boolean return value that indicates whether a user successfully invested in the project. The `investUpdate()` function should then use this boolean result to conditionally update the `whitelistRoots[_pid]` variable. This practice prevents state changes from occurring under invalid conditions.

8A2-15 | Inconsistent Soft Cap Evaluation

Category	Severity	Location	Status
Inconsistency	Minor	contracts/Fundraise.sol: 159, 246	Pending

Description

An inconsistency exists in the `Fundraise` contract regarding the evaluation of a project's soft cap. The `transferFundsToBorrower()` function implies a successful fundraise if `project.totalInvested == project.softCap` (L246). Conversely, the internal `_invest()` function requires `project.totalInvested` to strictly surpass `project.softCap` to transition the project to the `PreFunded` stage (L159). This discrepancy means that if a project precisely meets its soft cap, it will be considered successful for fund transfer but will fail to advance to the `PreFunded` stage.

Recommendation

It is recommended to change the condition in `_invest()` to include equality, ensuring that meeting the soft cap exactly also triggers the stage transition.

```
159  if (project.totalInvested >= project.softCap) {
```

8A2-16 | Missing Project Initialization Checks

Category	Severity	Location	Status
Access Control	Minor	contracts/Fundraise.sol: 337, 384	Pending

Description

The `createProject()` and `setProject()` functions in the `Fundraise` contract lack essential checks to ensure that new projects are initialized with correct default values and valid parameters. There are no checks to ensure that `totalInvested` and `totalRepaid` are zero when a new project is being set up, which are crucial for a fresh start. Furthermore, there are no validations to guarantee that `preFundDuration`, `softCap`, and `hardCap` are positive values, which are fundamental requirements for a viable fundraising project. This absence of validation allows for the creation or modification of projects with an inconsistent or invalid initial state.

Recommendation

It is recommended to implement comprehensive validation checks for `ComingSoon` projects within the `createProject()` and `setProject()` functions.

- Ensure `totalInvested` and `totalRepaid` are strictly zero when a new project is created or when these values are not explicitly intended to be updated with existing investment/repayment data.
- Validate that `preFundDuration`, `softCap`, and `hardCap` are greater than zero to ensure the project's viability.

8A2-17 | Inconsistent Token Decimals Assumption

Category	Severity	Location	Status
Volatile Code	Minor	contracts/RewardSystem.sol: 30	Pending

Description

The `RewardSystem` contract hardcodes a decimal value of 6 for USDC by default, which is not universally true across all blockchain networks (e.g., bridged USDC on BSC has 18 decimals). This incorrect assumption can lead to significant miscalculations in reward distributions.

Recommendation

Consider to dynamically fetch the decimal value from the USDC token contract instead of using a hardcoded value.

8A2-18 Incompatibility With Deflationary Tokens (Non-Standard ERC20 Token)

Category	Severity	Location	Status
Volatile Code	Informational	contracts/Fundraise.sol: 173, 278	Pending

Description

The project design may not be compatible with non-standard ERC20 tokens, such as deflationary tokens or rebase tokens.

The functions use `transferFrom()` / `transfer()` to move funds from the sender to the recipient but fail to verify if the received token amount matches the transferred amount. This could pose an issue with fee-on-transfer tokens, where the post-transfer balance might be less than anticipated, leading to balance inconsistencies. There might be subsequent checks for a second transfer, but an attacker might exploit leftover funds (such as those accidentally sent by another user) to gain unjustified credit.

Scenario

When transferring deflationary ERC20 tokens, the input amount may not equal the received amount due to the charged transaction fee. For example, if a user sends 100 deflationary tokens (with a 10% transaction fee), only 90 tokens actually arrive to the contract. However, a failure to discount such fees may allow the same user to withdraw 100 tokens from the contract, which causes the contract to lose 10 tokens in such a transaction.

Recommendation

We advise the client to regulate the set of tokens supported and add necessary mitigation mechanisms to keep track of accurate balances if there is a need to support non-standard ERC20 tokens.

8A2-19 `burnPercentage` Is Misleading Named

Category	Severity	Location	Status
Coding Style	Informational	contracts/RewardSystem.sol: 33, 214	Pending

Description

The variable `burnPercentage` in `RewardSystem` is misleadingly named, as it defaults to 6% but functions as a boolean trigger for a burn mechanism within the `activateProjectRewards()` function, rather than specifying a percentage of tokens to burn. This discrepancy between its name and actual usage can lead to confusion.

Recommendation

Consider changing the variable from `uint256 burnPercentage` to `bool burnTrigger` (or `isBurnEnabled`, `shouldBurn`, etc.).

8A2-20 Overly Restrictive Access Control For `makeRepayment()`

Category	Severity	Location	Status
Design Issue	Informational	contracts/Fundraise.sol: 274	Pending

Description

The `Fundraise::makeRepayment()` function is designed for project debt repayment, is overly restrictive by allowing only the project borrower or a designated contract manager to execute it. This stringent access control prevents external parties or "volunteers" from making repayments on behalf of a project, even if they are willing and able to do so. This limitation can hinder a project's ability to meet its repayment obligations, and reduces the flexibility of the protocol.

Recommendation

Consider removing the access control logic so that any `msg.sender` can call `makeRepayment()`, as long as they provide the correct funds and parameters.

APPENDIX | 8LENDS - AUDIT 2

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Access Control	Access Control findings are about security vulnerabilities that make protected assets unsafe.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Financial Manipulation	Financial Manipulation findings indicate issues in design that may lead to financial losses.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your Web3 Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is the largest blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.